

# Session 07 Teamwork — Build the Workspaces Feature

## Context

---

Work in your team (six teams of four students). The instructor's own `ProjectsModule` is already running — a complete feature with a service (data + lookup + a real update), a controller (routing), registered in `AppModule`. Your job is to build the same shape of feature, independently, for a second B4F Board resource: Workspaces.

## Goal

---

A working `WorkspacesModule`, following the exact same pattern as `ProjectsModule` — including the mutating endpoint — reasoned through by your team, not copied and renamed.

## What to build

---

- `WorkspacesService` (`@Injectable()`): a small, in-memory array of 2-3 workspaces, using the real B4F Board fields — `id`, `name`, `slug`, `description` — with:
  - `findAll()` and `findOne(id)` (`findOne` must throw `NotFoundException` when the `id` doesn't match).
  - `update(id, body)`: `400` if the new `name` is empty, `409` if the new `slug` is already used by another workspace, otherwise apply the change and return the updated workspace.
- `WorkspacesController` (`@Controller()` with the right prefix for this resource): `GET /workspaces`, `GET /workspaces/:id`, and `PATCH /workspaces/:id`.
- `WorkspacesModule` (`@Module()`): registers the controller and service.
- Register `WorkspacesModule` in `AppModule`'s `imports` array.

## Rules

---

**Allowed:** everything the instructor just used for Projects — `@Module`, `@Controller`, `@Get`, `@Param`, `@Patch`, `@Body`, `@Injectable`, constructor injection, and the same three built-in exceptions (`NotFoundException`, `BadRequestException`, `ConflictException`).

**Not yet:** no DTOs, no `class-validator`, no `ValidationPipe`, no guards, no database, no new npm package.

## Test it

---

BASH

```
curl http://localhost:3000/workspaces
curl http://localhost:3000/workspaces/1
curl http://localhost:3000/workspaces/999
curl -X PATCH http://localhost:3000/workspaces/1 -H "Content-Type: application/json" -d
'{"name":""}'
curl -X PATCH http://localhost:3000/workspaces/1 -H "Content-Type: application/json" -d
'{"name":"New Name"}'
```

## Done when

---

- `GET /workspaces` returns your full array.
- `GET /workspaces/:id` returns one workspace, or a `404` for an id that doesn't exist.
- `PATCH /workspaces/:id` returns `400` for an empty name, `409` for a slug already used by another workspace, and `200` with the updated workspace otherwise.
- Every team member can explain what each of the three new files is responsible for, and why the `update()` method checks things in the order it does, without re-reading them.

## Working time

---

Around 25 minutes.